

Projet : Synchronisation des Bus de Transport dans un Tunnel

Module : Systèmes d'exploitation 2

Niveau : L3 Informatique-ISIL

Année universitaire : 2024/2025

Réalisé par: LAKAS Lina Maria - HACI Amina

Question 2: Vérifier les quatre conditions de l'exclusion mutuelle. À rendre sur papier le jour du TP.

Notre solution est une implémentation correcte et équitable de la synchronisation des bus circulant dans un tunnel à voie unique, et le code fourni respecte les quatre conditions d'exclusion mutuelle :

1. Exclusion Mutuelle

L'exclusion mutuelle signifie qu'un seul processus (ou groupe d'une direction) peut accéder à la section critique (tunnel) à la fois.

Dans le code, l'exclusion mutuelle est assurée par le sémaphore `mutex` qui protège l'accès aux variables partagées (`insideX`, `insideY`, `waitingX`, `waitingY` et `turn`). De plus, la condition d'entrée dans le tunnel (`enter_tunnel`) vérifie qu'il n'y a pas de bus circulant dans le sens opposé avant d'autoriser l'entrée.

```
while (insideY > 0 || (turn == 1 && waitingY > 0)) {  
    // attente  
}
```

Cette condition garantit qu'aucun bus $X \rightarrow Y$ ne peut entrer si des bus $Y \rightarrow X$ sont déjà dans le tunnel, et vice versa.

Donc l'exclusion mutuelle est bien respectée. À tout moment, seuls des bus circulant dans la même direction peuvent se trouver dans le tunnel.

2. Absence d'Interblocage

Un interblocage se produit lorsqu'un groupe de bus (threads) attendent tous les uns sur les autres dans une dépendance circulaire, et qu'aucun ne peut progresser. Dans un système basé sur les sémaphores, cela peut arriver si les threads gardent des sémaphores tout en attendant d'autres ressources, ou si les ressources (comme l'accès au tunnel) sont retenues indéfiniment.

Dans notre code, l'interblocage est évité par le mécanisme de tour (`turn`) qui donne alternativement la priorité à l'une ou l'autre direction.

De plus, les bus ne retiennent jamais un sémaphore en attendant un autre, et chaque attente libère `mutex` avant de bloquer, ce qui brise toute possibilité de dépendance circulaire.

Quand un bus sort du tunnel et qu'il est le dernier de sa direction (`insideX == 0` ou `insideY == 0`), il change la valeur de `turn` et signale éventuellement un bus en attente dans l'autre direction :

```
if (insideX == 0) {
    turn = 1;
    if (waitingY > 0) {
        sem_post(&semY);
    }
}
```

Conclusion : Le code est exempt d'interblocage car le mécanisme de tour garantit qu'une direction ne peut pas attendre indéfiniment si l'autre est inactive.

3. Attente bornée (absence de famine)

La famine se produit lorsqu'un thread (un bus) est indéfiniment empêché d'accéder à une ressource. Dans ce code, le mécanisme de tour `turn` qui alterne la priorité entre les deux directions et assure l'équité: aucune direction ne peut monopoliser le tunnel pendant que l'autre direction attend indéfiniment.

Lorsque tous les bus d'une direction ont quitté le tunnel, la priorité est donnée à l'autre direction si des bus y sont en attente. Cette alternance de priorité garantit que chaque direction aura son tour, empêchant ainsi la famine et l'attente infinie.

```
// Si plus aucun bus X→Y à l'intérieur, donne la priorité aux Y→X
if (insideX == 0) {
    turn = 1;
    if (waitingY > 0) {
        sem_post(&semY);
    }
}
```

De plus, la condition d'entrée dans le tunnel (`enter_tunnel`) tient compte de la valeur de `turn` pour décider si un bus peut entrer :

```
while (insideY > 0 || (turn == 1 && waitingY > 0)) {
    // Un bus X→Y attend si le tour est à Y et qu'il y a des bus Y→X en attente
}
```

Donc le code prévient efficacement la famine grâce au mécanisme de tour qui alterne équitablement les priorités. L'alternance de priorité garantit que chaque direction aura son tour et assure l'attente bornée.

4. Progression

La progression signifie que si le tunnel est libre et qu'un bus attend, ce bus doit entrer dans le tunnel sans délai inutile, on doit aussi assurer que lorsqu'un bus attend, il ne peut pas bloquer les autres bus d'entrer dans le tunnel.

Dans ce code, si aucun bus n'est dans le tunnel (`insideX == 0` et `insideY == 0`), un bus en attente sera autorisé à entrer selon la valeur de `turn`.

Lorsqu'un bus est le dernier à quitter le tunnel dans sa direction, il signale un bus en attente dans l'autre direction, assurant ainsi le progrès.

```
if (insideY == 0) {
    turn = 0;
    if (waitingX > 0) {
        sem_post(&semX);
    }
}
```

Donc le code garantit le progrès en s'assurant qu'un bus en attente finira par recevoir l'autorisation d'entrer dans le tunnel lorsque celui-ci est libre.

Conclusion

Le code fourni respecte les quatre conditions d'exclusion mutuelle :

1. **Exclusion mutuelle** : Seuls des bus circulant dans la même direction peuvent être dans le tunnel simultanément.
2. **Absence d'interblocage** : Le mécanisme de tour empêche les situations où chaque direction attend l'autre indéfiniment.
3. **Attente bornée** : L'alternance de priorité garantit que chaque direction aura son tour.
4. **Progression** : Si le tunnel est libre, un bus en attente finira par y entrer.

Question 3: Cet algorithme peut-il être retenu comme solution pour protéger le tunnel? À rendre sur papier le jour du TP.

Oui, notre algorithme satisfait toutes les contraintes :

1. **Exclusion mutuelle** : Seuls des bus circulant dans la même direction peuvent être dans le tunnel simultanément (pas de croisement)
2. **Passage groupé autorisé** : Les bus $X \rightarrow Y$ peuvent entrer ensemble, les bus $Y \rightarrow X$ peuvent entrer ensemble, jamais les deux directions simultanément. (optimisation du débit)
3. **Équité garantie** : L'algorithme est équitable car il alterne systématiquement la priorité entre les directions $X \rightarrow Y$ et $Y \rightarrow X$, empêchant ainsi qu'une direction monopolise l'accès au tunnel. (alternance X/Y équilibrée)
4. **Allers-retours** : Chaque bus effectue 10 trajets aller-retour par jour, cela est garanti par l'utilisation de la variable `ALLER_RETOURS` dans la fonction `bus_thread` exécutée par chaque thread représentant une bus.